

FIAP — Faculdade de Informática e Administração Paulista
EV Challenge 2026 · Parceria FIAP + GoodWe

ChargeGrid Intelligence

Sistema Inteligente de Gerenciamento de Recarga

Documento Técnico — Sprint 2

Integrantes

Giovane Gomes Petenuci — RM 574091
Arthur Vettorazzo de Souza — RM 569445
Gustavo Zibini Belizario — RM 561376
Alan Junio Araujo de Souza — RM 574112
Brayan Barbosa Dos Santos — RM 573682
Luiz Otávio Brito Freixo — RM 569977

Hardware de referência: GoodWe GW11K-HCA-20 (11 kW AC, linha HCA G2)

Protocolo de integração: Modbus TCP

Segmento: eletropostos comerciais e de varejo

Sumário

1. Resumo Executivo.....	3
1.1. Números do projeto.....	3
2. Mapeamento dos Requisitos do Sprint 2.....	4
2.1. Entregáveis obrigatórios.....	4
2.2. Critérios de avaliação.....	4
3. Arquitetura do Sistema.....	6
3.1. Camadas e dependências.....	6
3.2. Responsabilidade de cada módulo.....	6
3.3. Justificativa da modularidade.....	7
4. Componentes e Regras de Negócio.....	8
4.1. Modelo de dados.....	8
4.2. Gerenciamento de múltiplas sessões.....	8
4.3. Controle de demanda de energia.....	8
4.4. Tarificação dinâmica.....	9
4.5. Simulação de integração — Modbus TCP.....	9
5. Interface e Fluxo Interativo.....	11
5.1. Fluxo de navegação.....	11
5.2. Telas e rotas.....	11
5.3. Funcionalidades de destaque.....	11
6. Cenários de Demonstração.....	13
7. Qualidade e Robustez.....	15
8. Como Executar.....	16
9. Conclusão.....	17

1. Resumo Executivo

O ChargeGrid Intelligence é uma solução de software para o ecossistema de mobilidade elétrica, desenvolvida no contexto do EV Challenge 2026 (parceria FIAP + GoodWe). O sistema resolve um problema concreto do segmento comercial e de varejo de eletropostos: a ausência de mecanismos integrados capazes de orquestrar potência elétrica, registrar ciclos de recarga, faturar sessões e comunicar informações ao usuário e à plataforma de gestão.

Esta entrega corresponde ao Sprint 2, cujo objetivo é um sistema capaz de gerenciar múltiplas sessões simultâneas, aplicar regras inteligentes de controle de potência, simular a integração com o hardware via protocolo industrial e gerar relatórios operacionais e financeiros.

A solução foi implementada como uma aplicação web em **Python (Flask)** no backend e **HTML/CSS/JavaScript** no frontend, com uma arquitetura modular em camadas que separa rigorosamente a interface das regras de negócio. O hardware de referência é o carregador AC GoodWe GW11K-HCA-20 (11 kW), e a integração é simulada através do protocolo **Modbus TCP**, utilizando os registradores reais da linha HCA G2.

1.1. Números do projeto

3.879	12	61	3 × 5
linhas de Python	rotas HTTP	testes automatizados	postos × conectores

Todos os 61 testes automatizados (pytest) passam. O sistema simula 3 postos de recarga independentes (Paulista, Faria Lima e Berrini), cada um com 5 conectores e limite de instalação de 33 kW, totalizando capacidade para 15 sessões simultâneas e 99 kW de carga gerenciada.

2. Mapeamento dos Requisitos do Sprint 2

Esta seção mapeia cada requisito obrigatório e cada critério de avaliação ao componente do sistema que o atende, para facilitar a verificação direta.

2.1. Entregáveis obrigatórios

Entregável	Onde está atendido	Status
Código em Python / JavaScript	Backend Flask (Python) + frontend (JS). 6 módulos de domínio.	Atendido
Execução com múltiplos cenários	Seção 6 deste documento + dashboard ao vivo com polling de 5s.	Atendido
Menu interativo	Navegação web: Mapa → Posto → Carregador → Dashboard (Seção 5).	Atendido
Documento técnico	Este documento.	Atendido

2.2. Critérios de avaliação

A tabela abaixo indica, para cada critério, o componente responsável e a seção deste documento onde a evidência é apresentada.

Critério (pontuação máxima)	Componente / Evidência	Seção
1. Gerenciamento de múltiplas sessões (20)	SessionManager isolado por posto; 3+ veículos simultâneos com controle organizado de ciclo de vida.	4.2 / 6
2. Controle de demanda de energia (20)	PowerManager com limite por instalação, throttle automático e redistribuição proporcional por prioridade.	4.3 / 6
3. Tarifação dinâmica (15)	PricingEngine com 3 eixos: horário de pico, demanda da rede e tipo de usuário.	4.4 / 6
4. Simulação de integração (15)	ModbusSimulator com frames TX/RX, registradores reais HCA G2 e envio/recebimento consistente.	4.5 / 6
5. Estrutura lógica (10)	Arquitetura modular em camadas, sem dependências circulares.	3
6. Interatividade (10)	Interface web responsiva com dark mode, toasts e atualização em tempo real.	5
7. Qualidade e robustez (10)	Tratamento de exceções, validação de entrada, thread-safety e 61 testes automatizados.	7

Observação sobre o “menu interativo”. Optou-se por uma interface web em vez de um menu de terminal. A navegação por telas (mapa de postos, lista de carregadores, formulário de sessão e painel central)

cumpre integralmente a função de menu interativo, oferecendo um fluxo mais próximo de um sistema real de operação de eletropostos — o que também atende ao critério de interatividade no nível mais alto.

3. Arquitetura do Sistema

O sistema adota uma arquitetura modular em camadas. Cada módulo tem uma responsabilidade única e bem definida, e as dependências fluem em uma única direção (das camadas superiores para as inferiores), sem ciclos. Essa separação cumpre a regra de projeto de nunca misturar rotas HTTP, regras de negócio e acesso a dados no mesmo arquivo.

3.1. Camadas e dependências

<code>models.py</code>	(camada base – entidades e enums; não depende de ninguém)
<code>session_manager.py</code>	(estado e ciclo de vida das sessões)
<code>power_manager.py</code>	(orquestracao de potencia: limite, throttle, rebalance)
<code>modbus_simulator.py</code>	(protocolo de integracao Modbus TCP)
<code>pricing_engine.py</code>	(tarifacao dinamica – depende apenas de models)
<code>logica_recarga.py</code>	(logica de simulacao do Sprint 1 – independente)
<code>app.py</code>	(camada web Flask – orquestra todos os modulos)

3.2. Responsabilidade de cada módulo

Módulo	Responsabilidade	Linhas
models.py	Entidade ChargingSession e enums SessionStatus e UserType. Núcleo de dados sem lógica de I/O.	204
session_manager.py	Criação, início, throttle, restauração e encerramento de sessões. Acúmulo de energia por tempo real.	397
power_manager.py	Controle de demanda: aloca potência, aplica throttle quando excede o limite e rebalanceia ao liberar capacidade.	494
pricing_engine.py	Tarifação dinâmica de três eixos (pico, demanda, tipo de usuário) e taxa mínima por sessão.	329
modbus_simulator.py	Simulação do protocolo Modbus TCP: frames TX/RX, registradores HCA G2 e log circular.	481
logica_recarga.py	Lógica de simulação minuto a minuto do Sprint 1, com tarifação proporcional ao tempo em pico.	273
app.py	Camada web Flask: 12 rotas HTTP, validação de entrada e orquestração dos módulos de domínio.	894

3.3. Justificativa da modularidade

A separação em módulos independentes não é apenas estética: ela viabiliza a evolução prevista para o Sprint 3, em que a persistência migrará para um banco de dados relacional (SQLAlchemy + PostgreSQL) e o simulador Modbus será substituído por comunicação com hardware real. Com a arquitetura atual, cada uma dessas trocas afeta um único módulo, sem impacto nos demais. A camada de domínio (regras de negócio) é totalmente independente da camada web e poderia, por exemplo, ser exposta também por uma API REST ou por uma interface de linha de comando sem qualquer reescrita.

4. Componentes e Regras de Negócio

4.1. Modelo de dados (models.py)

A entidade central é a `ChargingSession`, que representa uma sessão de recarga. Cada sessão possui um identificador único, o conector físico, a placa do veículo, o tipo de usuário, a potência solicitada e alocada, a energia acumulada, o custo e o status atual. O status é modelado por um enum explícito:

Status	Significado
CHARGING	Recarregando na potência integral solicitada.
THROTTLED	Recarregando com potência reduzida por controle de demanda.
COMPLETED	Sessão encerrada; custo final calculado.
FAULTED	Sessão recusada (capacidade física insuficiente).

Há também o enum `UserType`, com três categorias: `STANDARD` (Padrão), `SUBSCRIBER` (Assinante) e `CORPORATE` (Corporativo). O tipo de usuário influencia tanto a tarifa quanto a prioridade no controle de potência.

4.2. Gerenciamento de múltiplas sessões (session_manager.py)

O `SessionManager` controla todo o ciclo de vida das sessões. Cada posto possui sua própria instância, o que garante isolamento: o estado do posto Paulista nunca interfere no Faria Lima. As operações principais são:

- `create_session`: valida o conector e a potência solicitada (faixa de 4,2 a 11 kW) e cria a sessão; rejeita conectores já ocupados.
- `start_charging`: inicia o fornecimento de energia na potência alocada e fixa a tarifa do momento da conexão.
- `accrue_energy`: acumula energia com base no tempo real decorrido desde a última atualização, sendo idempotente — múltiplos acessos (polling, atualização de página) convergem para o mesmo valor real, sem inflar o consumo.
- `finish_session`: encerra a sessão, aplica a taxa mínima e libera o conector. É idempotente: chamar duas vezes não reprocessa.

Decisão de robustez. O cálculo de energia por tempo real corrige um problema clássico de simulações ingênuas, em que a energia cresceria a cada requisição HTTP. Aqui, a energia depende apenas do relógio, tornando o valor consistente independentemente de quantas vezes a interface é atualizada.

4.3. Controle de demanda de energia (power_manager.py)

O `PowerManager` é o componente responsável pelo controle inteligente de potência — o critério de maior peso na avaliação. Cada posto tem limite de instalação de 33 kW. Como cada conector pode entregar até 11 kW, três veículos simultâneos ($3 \times 11 = 33$ kW) cabem exatamente no limite. A partir do quarto veículo, a carga projetada (44 kW) excederia a capacidade da instalação, e o sistema reage automaticamente.

Algoritmo de alocação

Ao receber uma nova sessão, o PowerManager compara a carga projetada com o limite:

- Se **cabe no limite**: concede a potência integral de 11 kW (status CHARGING).
- Se **excede o limite**: redistribui a potência disponível entre todas as sessões, reduzindo-as proporcionalmente (status THROTTLED). Nenhuma sessão fica abaixo do piso físico de 4,2 kW.

Prioridade por tipo de usuário

A redistribuição não é igualitária: usuários premium recebem fatias maiores, refletindo o modelo de negócio. Os pesos aplicados são:

Tipo de usuário	Peso	Vantagem
Padrão (STANDARD)	1,00	base
Assinante (SUBSCRIBER)	1,20	+20%
Corporativo (CORPORATE)	1,10	+10%

Rebalanceamento. Quando um veículo encerra a sessão e libera capacidade, o método rebalance restaura potência às sessões remanescentes — também respeitando a prioridade por tipo. Uma sessão só retorna ao status CHARGING quando recupera integralmente a potência solicitada; caso contrário, permanece THROTTLED com a potência mais alta possível. O algoritmo recalcula a capacidade disponível a cada restauração, tornando matematicamente impossível ultrapassar o limite da instalação.

4.4. Tarifação dinâmica (pricing_engine.py)

O PricingEngine calcula a tarifa por kWh combinando três eixos independentes, satisfazendo o nível máximo do critério (dinâmica por horário, demanda e tipo):

Eixo	Fator	Condição de aplicação
Tarifa base	R\$ 1,20/kWh	Sempre (ponto de partida).
Horário de pico	× 1,50	Entre 18h00 e 22h59.
Alta demanda	× 1,30	Ocupação do posto ≥ 70%.
Desconto Assinante	– 15%	Usuário do tipo SUBSCRIBER.
Desconto Corporativo	– 10%	Usuário do tipo CORPORATE.
Taxa mínima	R\$ 2,00	Piso de cobrança por sessão.

Decisão de design. A tarifa é fixada no instante da conexão e permanece constante durante a sessão — prática comercial padrão equivalente a uma tarifa contratada: o usuário sabe exatamente quanto pagará por kWh ao conectar. O eixo de demanda usa a ocupação do posto específico (não a média global da rede), de modo que a tarifa de escassez reflete a real congestão do local onde o veículo está conectado.

4.5. Simulação de integração — Modbus TCP (modbus_simulator.py)

A integração com o hardware é simulada pelo protocolo **Modbus TCP**, que é o protocolo real do carregador GoodWe GW11K-HCA-20 (linha HCA G2) — confirmado em mentoria com a empresa. Cada

evento do ciclo de recarga gera frames Modbus consistentes, com par envio/recebimento (TX/RX), endereço de registrador, código de função e payload.

Os registradores utilizados são os reais do equipamento, conforme a documentação do protocolo HCA G2:

Registrador	Função	Acesso
10017	Status da estação de recarga	Leitura
10015	Potência de recarga ($\div 10 = \text{kW}$)	Leitura
10016	Energia da sessão ($\div 10 = \text{kWh}$)	Leitura
10029	Potência máxima de recarga	Leitura/Escrita
10025	Gestão dinâmica de carga	Leitura/Escrita
10060	Liga/desliga recarga (1=off, 2=on)	Leitura/Escrita
10026	Corrente do disjuntor (A)	Leitura/Escrita

Os eventos que disparam comunicação Modbus são: início de sessão (escreve potência máxima, habilita recarga, lê status), throttle (reescreve a potência máxima dos conectores afetados), leitura periódica de medidores (potência e energia instantâneas, equivalente ao MeterValues do OCPP), rebalanceamento e encerramento. O log de frames é mantido em um buffer circular de 500 entradas, evitando crescimento ilimitado de memória durante sessões longas, enquanto um contador acumulado preserva a métrica histórica total.

5. Interface e Fluxo Interativo

A interface web é o “menu interativo” do sistema. O fluxo de navegação foi desenhado para refletir a operação real de uma rede de eletropostos, indo do panorama geral ao detalhe de cada sessão.

5.1. Fluxo de navegação

[Mapa de Postos]	-> visao geral dos 3 postos, com status de cada um
[Posto]	-> os 5 conectores do posto; throttle sinalizado
[Carregador]	-> formulario de nova sessao (Sprint 1)
[Dashboard]	-> painel central: cria/encerra sessoes, tempo real
[Relatorio]	-> consolidado de sessoes + snapshot Modbus
[Log Modbus]	-> frames TX/RX do protocolo
[Testes]	-> execucao da suite pytest no navegador

5.2. Telas e rotas

Tela	Descrição	Rota
Mapa de postos	Pontos clicáveis dos 3 postos, com indicação de throttle.	/
Carregadores do posto	Os 5 conectores (C1–C4 públicos, C5 VIP/assinante).	/posto/<id>
Painel central (Dashboard)	Potência por posto, sessões ativas, tarifas e decisões de potência, com atualização a cada 5 segundos.	/dashboard
Relatório consolidado	Todas as sessões, receita realizada e projetada, snapshot dos registradores.	/relatorio
Log Modbus	Frames TX/RX com endereço, função e payload.	/modbus-log
Testes	Executa a suíte pytest e exibe o resultado no navegador.	/testes

A interface conta com design responsivo, modo escuro, notificações (toasts) e um painel de potência que mostra simultaneamente o agregado da rede (99 kW) e a telemetria individual de cada posto (33 kW), com barras coloridas por nível de ocupação (verde, ambar, vermelho).

5.3. Funcionalidades de destaque

Mapa interativo (Leaflet + OpenStreetMap). Os três postos são exibidos sobre um mapa real de São Paulo, posicionados em suas coordenadas geográficas verdadeiras (Av. Paulista, Av. Faria Lima e Av.

Berrini). Os marcadores mudam de cor conforme o estado — verde (disponível), âmbar (controle dinâmico ativo) e vermelho (lotado, 100% ocupado) — e abrem um cartão com nome, endereço, status e acesso aos carregadores. Caso não haja conexão com a internet, um aviso é exibido e a navegação pela lista lateral permanece funcional.

Atualização em tempo real. O painel e o relatório atualizam minutos de recarga, energia consumida, custo e potência automaticamente, sem recarregar a página. Como o cálculo de energia depende do relógio do servidor, os valores crescem de forma contínua e consistente enquanto a sessão está ativa.

Conector VIP exclusivo. O quinto conector (C5) de cada posto é reservado a assinantes. Usuários Padrão e Corporativo são impedidos de iniciar sessão nesse conector, com mensagem explicativa. O filtro “Assinantes” no mapa destaca os postos com o C5 livre e marca os demais como indisponíveis, sem ocultá-los.

Ambiente de demonstração pronto. Ao iniciar, o sistema já popula automaticamente cenários representativos: um posto lotado e em throttle, e outro com múltiplas sessões e o conector de assinante livre. Isso permite visualizar de imediato os estados de ocupação, throttling e prioridade, sem configuração manual.

6. Cenários de Demonstração

Esta seção apresenta cenários reais executados no sistema, com os números efetivamente produzidos. Eles demonstram, em sequência, o gerenciamento de múltiplas sessões, o controle de demanda, a tarifação dinâmica e o rebalanceamento.

6.1. Cenário 1 — Três veículos dentro do limite

Três veículos de tipos diferentes conectam-se ao posto Paulista às 10h (fora do horário de pico). A carga projetada ($3 \times 11 = 33$ kW) cabe exatamente no limite, então todos recarregam na potência integral.

Conector	Tipo	Potência	Status
P1-C1	Padrão	11,0 kW	CHARGING
P1-C2	Assinante	11,0 kW	CHARGING
P1-C3	Corporativo	11,0 kW	CHARGING

Resultado: carga total de 33,0 kW (100% do posto), nenhuma sessão throttled.

6.2. Cenário 2 — Cinco veículos e controle de demanda

Mais dois veículos conectam-se ao mesmo posto, totalizando cinco. A carga projetada (55 kW) excede o limite, e o controle de demanda reduz automaticamente todas as sessões, respeitando a prioridade por tipo de usuário.

Conector	Tipo	Potência alocada	Status
P1-C2	Assinante	7,37 kW	THROTTLED
P1-C3	Corporativo	6,75 kW	THROTTLED
P1-C5	Assinante	6,60 kW	THROTTLED
P1-C1	Padrão	6,14 kW	THROTTLED
P1-C4	Padrão	6,14 kW	THROTTLED

Resultado: carga mantida em exatamente 33,0 kW. Note a prioridade: o Assinante (7,37 kW) recebe mais que o Corporativo (6,75 kW), que recebe mais que os Padrão (6,14 kW). O limite da instalação nunca é violado.

6.3. Cenário 3 — Tarifação dinâmica

A tabela mostra a tarifa por kWh para cada tipo de usuário em três condições: normal (fora de pico, baixa demanda), pico (19h) e pico com alta demanda do posto ($\geq 70\%$). Os três eixos se compõem multiplicativamente.

Tipo	Normal	Pico	Pico + Demanda
Padrão	R\$ 1,2000	R\$ 1,8000	R\$ 2,3400
Assinante (-15%)	R\$ 1,0200	R\$ 1,5300	R\$ 1,9890
Corporativo (-10%)	R\$ 1,0800	R\$ 1,6200	R\$ 2,1060

Exemplo de composição: Padrão em pico com alta demanda = $1,20 \times 1,50 \times 1,30 = \text{R\$ } 2,34/\text{kWh}$. Assinante na mesma condição recebe ainda o desconto de 15%: $2,34 \times 0,85 = \text{R\$ } 1,9890/\text{kWh}$.

6.4. Cenário 4 — Encerramento e rebalanceamento

Um dos cinco veículos (um Padrão) encerra a recarga. O sistema rebalanceia automaticamente, redistribuindo a capacidade liberada entre os quatro remanescentes — preservando a prioridade por tipo.

Conector	Tipo	Nova potência	Status
P1-C2	Assinante	8,80 kW	THROTTLED
P1-C5	Assinante	8,80 kW	THROTTLED
P1-C3	Corporativo	8,07 kW	THROTTLED
P1-C4	Padrão	7,33 kW	THROTTLED

Resultado: as quatro sessões sobem de potência (a capacidade liberada foi redistribuída), a prioridade por tipo é mantida e a carga total permanece em 33,0 kW. Como nenhuma sessão atingiu os 11 kW solicitados, todas continuam corretamente no status THROTTLED.

6.5. Evidência da simulação Modbus

Durante os cenários acima, o simulador Modbus gerou frames consistentes para cada evento. Um resumo dos tipos de frame produzidos:

Tipo de frame Modbus	Evento associado
Definição de potência máxima (reg. 10029)	Início e throttle
Habilitação de recarga (reg. 10060)	Início de sessão
Ativação de gestão dinâmica (reg. 10025)	Throttle
Leitura de status / potência / energia	Medição periódica
Confirmação de status pós-throttle	Redistribuição

Cada frame inclui identificador de transação, código de função, endereço de registrador e payload com o valor codificado (por exemplo, 11,0 kW é codificado como 0x006E, ou seja, 110 em décimos de kW), demonstrando uma simulação consistente de envio e recebimento — o nível máximo do critério.

7. Qualidade e Robustez

Por lidar com energia, faturamento e integração com hardware, o sistema foi construído com forte ênfase em robustez. As principais garantias são:

7.1. Tratamento de exceções e validação de entrada

- Validação de potência: a faixa aceita é 4,2 a 11 kW; valores fora disso são rejeitados com mensagem clara, sem derrubar o sistema.
- Validação de placa: aceita os formatos brasileiros antigo (ABC1234) e Mercosul (ABC1D23), normalizando para maiúsculas; placas inválidas não criam sessão.
- Validação de horário: a hora de início é validada na faixa de 0 a 23.
- Proteção de conector: um conector já ocupado não pode receber uma segunda sessão simultânea.
- Idempotência: encerrar uma sessão duas vezes, ou acumular energia múltiplas vezes, não corrompe o estado nem o faturamento.

7.2. Concorrência (thread-safety)

As operações críticas de criação, encerramento e acúmulo de energia são protegidas por um lock de exclusão mútua. Um teste com 10 threads concorrentes confirmou que o sistema mantém a integridade: no máximo 5 sessões por posto, carga total exata de 33,0 kW e nenhuma condição de corrida.

7.3. Testes automatizados

O projeto inclui 61 testes automatizados (pytest), todos passando, organizados por área:

Suíte de testes	Casos	Cobertura
Tarificação (PricingEngine)	13	Os três eixos e fronteiras de horário.
Alocação de potência (PowerManager)	6	Limites, throttle e recusa física.
Rebalanceamento	4	Restauração e prioridade por tipo.
Ciclo de vida (SessionManager)	8	Energia real, idempotência, taxa mínima.
Rotas Flask	8	Todas as rotas, incluindo posto lotado.
Regressão e Auditoria	18	Não-reincidência dos bugs corrigidos.
Conector VIP	4	Exclusividade do conector C5 para assinantes.

A suíte pode ser executada diretamente pelo navegador, na rota /testes, exibindo o resultado em tempo real — um recurso adicional de transparência para a avaliação.

8. Como Executar

O sistema requer Python 3 e duas bibliotecas. Os passos para execução local são:

```
# 1. Instalar as dependencias
pip install flask pytest

# 2. Iniciar a aplicacao
python app.py

# 3. Acessar no navegador
http://localhost:5001

# (Opcional) Rodar os testes automatizados
pytest test_chargegrid.py -v
```

Ao iniciar, a aplicação imprime no terminal o endereço local e o endereço de rede, permitindo o acesso a partir de outros dispositivos na mesma rede. A suíte de testes também pode ser executada pela rota /testes na própria interface.

8.1. Estrutura de arquivos entregue

chargeGrid_sprint2/	
app.py	- aplicacao Flask (ponto de entrada)
models.py	- entidades e enums
session_manager.py	- ciclo de vida das sessoes
power_manager.py	- controle de demanda de potencia
pricing_engine.py	- tarifacao dinamica
modbus_simulator.py	- simulacao do protocolo Modbus TCP
logica_recarga.py	- logica de simulacao do Sprint 1
test_chargegrid.py	- 61 testes automatizados
templates/	- 9 telas HTML (interface web)

9. Conclusão

O ChargeGrid Intelligence cumpre integralmente os objetivos do Sprint 2: gerencia múltiplas sessões simultâneas de forma organizada e isolada por posto, aplica controle inteligente de demanda com redistribuição proporcional por prioridade, calcula tarifação dinâmica em três eixos e simula de forma consistente a integração com o hardware via Modbus TCP, além de gerar relatórios operacionais e financeiros.

A arquitetura modular, a cobertura de 61 testes automatizados e o tratamento rigoroso de erros aproximam o sistema de um produto real, e não de um mero protótipo acadêmico. Cada módulo tem responsabilidade única e bem definida, com a camada de regras de negócio totalmente independente da camada web.

A modularidade adotada é o principal alicerce do projeto: ela mantém o código legível, testável e preparado para evoluir, permitindo que componentes específicos sejam substituídos ou ampliados sem exigir a reescrita das regras de negócio já validadas neste Sprint.